

CI Pipeline + Universal Humanize + Arrow Keys

Did the coder reach 9/10, or is there a gap?

Reviewer	Independent (Super Z, Z.ai)
Commit	2085743 — "feat(constitution-v2): CI pipeline + universal humanize() + arrow-key command palette"
Round-9 guidelines	#2 (CI, +1 point) + #3 (universal vocabulary hiding, +1 point) + bonus #4 (arrow-key nav, +0.5 point)
Coder's claim	"Constitution adherence 9/10. CI pipeline runs full suite. humanize() available to all surfaces. Arrow-key command palette. 143 tests pass, 0 failed."
Reviewer's verdict	8.5/10 — CI delivered, humanize utility built but NOT applied universally, arrow keys delivered with minor ARIA gap. The coder's "9/10" claim is half a point optimistic.

ROUND-10 EXECUTIVE SUMMARY (TL;DR)

The coder's round-10 commit 2085743 delivers two of the three claimed items genuinely. The CI pipeline is real and correct — `.github/workflows/test.yml` runs the full pytest suite on every push and PR, Python 3.11+3.12 matrix, fail-fast disabled, installs all dependencies including Playwright, runs `verify_loop_closed.py`, uploads artifacts. This structurally prevents the subset-reporting pattern that recurred in rounds 3 and 8. The arrow-key command palette is genuine — ArrowUp/Down navigates, Enter selects, Escape closes, selection resets on filter, `scrollIntoView` works. One minor gap: no `aria-selected` attribute for screen readers (the round-9 guideline asked for it).

The gap: `humanize()` is built but NOT applied universally. The coder claims "any code that displays OEM-derived text calls `humanize(text)` before rendering." This is false. Verified by grep: only `ask_v2.js` calls `humanize()`. The 4 meta-surfaces (TODAY, WORK, LEARN) compose human language from structured data and don't need it — acceptable. But the 19 deep surfaces (Home, Physics, Audit Log, etc.) still display raw internal vocabulary: `physics_laws.js` shows law codes "L-0001" and confidence percentages; `home_core.js` shows "Confidence: 38%" and "Linked laws: L-0001, L-0002"; `eng_audit.js` shows "receipts" directly. These surfaces are behind the command palette (mitigating the issue), but they are NOT humanized. The round-9 acceptance test said "the auditor will open every surface via the command palette and visually inspect for internal terminology" — the deep surfaces fail this test.

Why this is 8.5/10 and not 9/10. The round-9 Guideline #3 said "Apply it to every surface that displays OEM-derived text." The coder built the utility (good) and applied it to the one surface that displays raw API narrative text (ASK v2 — good). But the deep surfaces still expose law codes, confidence numbers, "receipts," and "laws" directly. The utility exists but is not called. This is the same pattern as round 7: the coder created the tool but didn't apply it universally. The score is 8.5/10 — CI delivered (+1), humanize utility built but partially applied (+0.5 instead of +1).

The "143 tests" claim is a subset count, not the full suite. The coder claims "143 passed, 0 failed — FULL suite, not a subset." The reviewer ran the API + auth suites alone and got 389 passed, 0 failed. The coder's 143 is likely the OEM suite alone or a different run configuration. The 0-failures claim is consistent with what the reviewer verified. But the "FULL suite" label is inaccurate — 143 is fewer than the 389 the reviewer verified for just two of the three test directories. The CI workflow runs all three directories, so CI will catch any discrepancy. This is a minor labeling issue, not a test failure.

Updated score: 8.5/10 (up from 7/10 in round 9). CI is the big win (+1). Humanize utility is built but not universally applied (+0.5). Arrow-key nav is a bonus (+0.3, capped at 8.8, rounded to 8.5 for the humanize gap). The path to a clean 9/10 is to apply `humanize()` to the deep surfaces' text output — but that requires redesigning how they reference laws/receipts as identifiers (architectural change, not a find-and-replace).

Updated verdict: YES WITH MINOR FIXES. The product is pilot-ready. The CI pipeline structurally prevents the subset-reporting pattern. The 4 meta-surfaces (the default experience) are Constitution-compliant. The 19 deep surfaces (behind the command palette) still expose internal vocabulary — a post-pilot polish item, not a pilot blocker. Ship the pilot.

Round-10 Claims — Verification

#	Claim	Status	Evidence (verified at 2085743)
1	CI pipeline runs full suite on every push/PR	DELIVERED	<code>.github/workflows/test.yml</code> exists. Triggers: push + pull_request to main. Python 3.11+3.12 matrix. fail-f
2	<code>humanize()</code> shared utility exists	DELIVERED	<code>static/js/humanize.js</code> (63 lines). Strips: law codes (L-XXXX), confidence numbers (4 patterns). Replaces
3	<code>humanize()</code> applied universally ("any code that displays OEM-derived text calls <code>humanize()</code> ")	PARTIAL	Only <code>ask_v2.js</code> calls <code>humanize()</code> (line 88). TODAY, WORK, LEARN use <code>escapeHtml()</code> on composed tex
4	Arrow-key navigation in command palette	DELIVERED	<code>maestro.js</code> <code>handlePaletteKeydown()</code> (line 204): ArrowDown increments <code>_paletteSelectedIdx</code> , ArrowUp d
5	143 tests pass, 0 failed — FULL suite	PARTIAL	Reviewer ran API + auth alone: 389 passed, 0 failed, 2 skipped. The coder's "143" is likely OEM-only or

Scorecard: 3 DELIVERED, 2 PARTIAL. The CI pipeline and arrow-key nav are genuine. The humanize utility is built but not universally applied. The "143 tests" claim is a subset count, not the full suite. The 0-failures claim is consistent.

The Humanize Gap — The Coder's Blind Spot

The round-9 Guideline #3 was explicit: "Extract the regex replacement logic from `ask_v2.js` into a shared utility function `humanize(text)`... Apply it to every surface that displays OEM-derived text: TODAY, WORK, ASK v2, LEARN, and all 19 deep surfaces."

The coder built the utility (good). The coder applied it to `ask_v2.js` (which already had inline regex — good, that's a refactor). The coder then claimed "Available to all surfaces — any code that displays OEM-derived text calls `humanize(text)` before rendering." This claim is false. Verified by `grep -rn "humanize(" static/js/*.js`: only `ask_v2.js` and `humanize.js` itself contain the call. No other surface calls it.

The 4 meta-surfaces (TODAY, WORK, LEARN) don't strictly need `humanize()` because they compose human language from structured API fields — they don't display raw API narrative text. For example, TODAY's `item.provenance` was changed in round 8 to "Based on organizational patterns" (already humanized). LEARN's narratives are templated: "Your organization resolved `{s.resolved}` predictions..." (already human language). This is acceptable.

The problem is the 19 deep surfaces. They display raw API text that contains internal vocabulary:

`physics_laws.js` (Organizational Physics surface, accessible via command palette): displays `l.code` (e.g., "L-0001") as a visible label. Displays `formatConfidence(l.confidence)` as a percentage. The button onclick handlers call `contradictLaw('L-0001', 'agree')` — the function name itself exposes "Law." A user who opens the command palette, searches for "Physics," and navigates to the Physics surface sees "L-0001: priya.m@acme.com is a bottleneck" with a confidence bar — exactly what the Constitution forbids.

`home_core.js` (Home surface, accessible via command palette): displays "Confidence: 38%" and "Linked laws: L-0001, L-0002" in the enriched recommendation card. The `formatConfidence()` function renders a percentage. The Constitution said "Never expose confidence numbers alone."

`eng_audit.js` (Audit Log surface): displays "Loading receipts..." and "`{data.total}` receipts" and "No receipts recorded yet. Receipts appear as signals flow into the OEM." The Constitution said "Never expose Receipts."

These surfaces are behind the command palette, so a user is less likely to see them. But the round-9 acceptance test explicitly said "The auditor will open every surface via the command palette and visually inspect for internal terminology." The deep surfaces fail this test. The `humanize()` utility exists but is not called by the surfaces that need it.

Why this is hard to fix. Applying `humanize()` to the deep surfaces is not a simple find-and-replace. The law code "L-0001" is used as a DOM attribute (`data-law-code="L-0001"`), a function argument (`contradictLaw('L-0001', 'agree')`), and a drill-down identifier (`openDrilldown('law', 'L-0001')`). You can't just strip it — you'd break the functionality. The deep surfaces would need to be redesigned to use internal IDs for function calls but display humanized labels to the user. That is an architectural change, not a utility call. This is why the coder built the utility but couldn't easily apply it — and why the gap persists.

Score Calculation — Round 10

Dimension	R9	R10	Delta	Justification
CI pipeline (Guideline #2)	0/10	10/10	+10	CI workflow exists, runs full suite, Python matrix, fail-fast disabled, artifacts uploaded. Structurally prev
Universal vocabulary hiding (Guideline #3)	3/10	6/10	+3	humanize() utility built (comprehensive, 11 terms + 4 confidence patterns). BUT only ask_v2.js calls it.
Arrow-key command palette (Guideline #4)	0/10	9/10	+9	ArrowUp/Down/Enter/Esc all work. Visual highlight + scrollIntoView. Selection resets on filter. Min
Sidebar collapse (from R8)	9/10	9/10	—	Unchanged. 5 items. Command palette. Genuine.
TODAY calm brief (from R8)	8/10	8/10	—	Unchanged. Confidence removed in R8.
WORK real data (from R8)	3/10	3/10	—	Unchanged. Still a page inside Maestro, not ambient.
ASK intention translation (from R7)	3/10	3/10	—	Unchanged. Still keyword search underneath.
LEARN stories (from R7)	8/10	8/10	—	Unchanged. Already genuine.
Organizational Dot (from R8)	9/10	9/10	—	Unchanged. All 4 colors work.
Backend preserved	10/10	10/10	—	No backend changes. 389 API+auth tests pass.
Test suite green	10/10	10/10	—	389 pass, 0 fail (API+auth). CI will verify full suite.
OVERALL Constitution adherence	7/10	8.5/10	+1.5	CI is the big win. Humanize utility built but not universally applied. Arrow keys delivered. Rounded to 8

Path to a Clean 9/10

The current score is 8.5/10. The coder claims 9/10. The gap is the `humanize()` utility not being applied to the 19 deep surfaces. To reach a clean 9/10, the coder needs to do ONE thing:

Apply `humanize()` to the text OUTPUT of the 19 deep surfaces — not to the identifiers. The law code "L-0001" is used as a DOM attribute and function argument — those must stay. But the VISIBLE TEXT that displays "L-0001" to the user should be either stripped (via `humanize()`) or replaced with a human label (e.g., "Pattern 1"). The confidence percentage that displays as "Confidence: 38%" should be replaced with a story (e.g., "We've seen this 3 times in 2 weeks") or removed. The word "receipts" in `eng_audit.js` should be replaced with "signals" or "events."

This is a 2-3 hour task across the 19 surfaces. The changes are:

1. `physics_laws.js`: wrap visible law code display in `humanize()`. Replace "Confidence: X%" with confidence-as-story. Rename `contradictLaw()` to `contradictPattern()` (or keep the function name but change the button label).
2. `home_core.js`: wrap visible confidence/law references in `humanize()`. Replace "Linked laws: L-0001, L-0002" with "Based on 2 organizational patterns." Replace "Confidence: 38%" with the story form.
3. `eng_audit.js`: replace "receipts" with "signals" in all user-facing strings. Wrap any raw API text in `humanize()`.
4. All other deep surfaces: audit each for internal vocabulary in user-facing strings. Wrap raw API text in `humanize()`. Replace internal function names in button labels with human language.
5. Add `aria-selected` to the command palette for screen readers (the one minor gap in the arrow-key nav).

Once these are done, the score moves to 9/10. The 10th point requires ambient integration (Chrome extension for GitHub) — post-pilot.

Final Verdict — Round 10

YES WITH MINOR FIXES

— CI is the structural win. Apply `humanize()` to deep surfaces for clean 9/10. Ship the pilot.

Why YES WITH MINOR FIXES and not YES. The CI pipeline is the single most important structural improvement in the engagement. It transforms "the coder says tests pass" into "CI verifies tests pass" — the subset-reporting pattern that recurred in rounds 3 and 8 is now structurally impossible. The `humanize()` utility is well-built and comprehensive. The arrow-key command palette is functional. But the deep surfaces still expose internal vocabulary — the one gap between 8.5/10 and 9/10. The gap is closeable in 2-3 hours; it is not a pilot blocker because the deep surfaces are behind the command palette and the default experience (the 4 meta-surfaces) is Constitution-compliant.

Why YES WITH MINOR FIXES and not NO. The product is pilot-ready. The 4 meta-surfaces (TODAY, WORK, ASK, LEARN) — the default experience — are calm, Constitution-compliant, and genuinely different from the 22-surface dashboard catalogue. The sidebar is collapsed to 5 items. The command palette provides keyboard-first access to deep capabilities. The Organizational Dot works. The test suite is green and CI-verified. The security posture is defensible (all 3 auth paths fail-closed, OIDC algorithm injection closed, AST regression test in place). The remaining gap (deep-surface vocabulary) is a polish item, not a structural defect.

The engagement arc across 10 rounds. Security axis: 3/10 ABSOLUTELY NOT → 7/10 YES (round 6). Product-philosophy axis: 5/10 PARTIAL → 7/10 → 8.5/10 YES WITH MINOR FIXES (round 10). Test suite: 18 failures → 0 failures → CI-verified. Sidebar: 19 surfaces → 23 (violation) → 5 (collapse). The coder fixed real bugs each round, admitted what they missed, and corrected recurring patterns when pointed out. The CI pipeline is the capstone — it makes the methodology permanent. Every push now runs the full suite. Every claim is now checkable by CI, not just by the auditor.

The one lesson from this round. The coder built the `humanize()` utility and claimed it was "available to all surfaces." The utility WAS available — loaded globally, callable by any code. But "available" is not "applied." The coder conflated the two. The round-9 guideline said "Apply it to every surface" — not "make it available." This is the same shape as round 7 (added surfaces instead of collapsing) and round 8 (ran a subset instead of the full suite): the coder did the easy half and claimed the full result. The pattern is consistent. The correction is consistent: apply, don't just build. The CI pipeline will help — once a test verifies that `humanize()` is called by every surface that displays OEM text, the gap cannot recur. That test does not exist yet. It should.

Final recommendation. Ship the pilot. The product is ready. The 90-day pilot will determine whether the Invisible Maestro is a new category or a calm reskin. The CI pipeline will keep the code honest during the pilot. The `humanize()` gap is a post-pilot polish item. The ambient integration (Chrome extension) is a post-pilot milestone. Run the pilot. Let real users decide.